COL7524 Advanced Networks: Assignment 3, Software-Defined Networking

Entry numbers: 2026MCS2493 · 2026MCS2494

Environment and how to run

Component	Version	Component	Version
OS	Windows 11 + WSL2, Ubuntu 24.04 LTS	Ryu	4.34 (Python 3.9 venv `~/ryu-env`)
Mininet	2.3.0 (system Python 3.12)	eventlet / setuptools	0.30.2 / 67.6.1 (pinned, newer break Ryu)
Open vSwitch	3.3.9, OpenFlow 1.3	iperf	2.1.9

The controller always listens on `127.0.0.1:6633`. Run commands are also written at the top of every script.

```
sudo service openvswitch-switch start                    # after every WSL restart
# Part 1 (two terminals)
source ~/ryu-env/bin/activate; ryu-manager --ofp-tcp-listen-port 6633 p1_reactive.py   # or p1_proactive.py
sudo mn -c; sudo python3 p1_topo.py
# Part 2 and Part 3 (one command each: starts Ryu, runs Mininet scenario, saves log + output)
cd "Part 2"; sudo bash run_p2.sh default ; sudo bash run_p2.sh changed
cd "Part 3"; sudo bash run_p3.sh aware   ; sudo bash run_p3.sh static
```

Folder	Contents
`Part 1/`	`p1_reactive.py`, `p1_proactive.py`, `p1_topo.py`, Ryu logs, Mininet session recordings
`Part 2/`	`p2_controller.py`, `p2_topo.py`, `links_default.json`, `links_changed.json`, `run_p2.sh`, `p2_experiment.py`, logs
`Part 3/`	`p3_controller.py` (extends Part 2), `run_p3.sh`, `p3_experiment.py`, logs. Uses `../Part 2`, so keep both folders together.

Topology changes

- **Part 1:** `p1_topo.py` unchanged.
- **Part 2:** `p2_topo.py` builds the same ring as `p3_topo.py`, but reads each ring link's bandwidth and delay from a JSON file so that topology and controller use the same values. With `links_default.json` it is identical to `p3_topo.py`; `links_changed.json` only raises the s1–s2 delay from 20 ms to 40 ms.
- **Part 3:** same ring and link values, but the ring-link queues are limited to 100 packets (Mininet default: 1000). At 1 Mbit/s, 1000 packets is about 12 s of buffering, which made TCP results and sender-side iperf numbers meaningless.

Part 1: Reactive and proactive flow installation

Topology: `p1_topo.py`, s1 with h1–h3, s2 with h4–h5, s1–s2 link 10 Mbit/s, 5 ms. Port map: s1: h1=1, h2=2, h3=3, s2=4; s2: h4=1, h5=2, s1=3.

Design

- **Reactive (`p1_reactive.py`):** a MAC-learning switch. When a switch connects, only a table-miss rule (send to controller) and an IPv6 drop rule are installed. On every Packet-In the controller learns *source MAC → in_port*. If the destination is already known it installs `(in_port, eth_src, eth_dst) → out_port` (priority 1, idle timeout 60 s); otherwise it floods. The packet itself is always sent on with a Packet-Out.

- **Proactive (`p1_proactive.py`):** the topology is known in advance. When a switch connects, the controller installs table-miss, IPv6 drop, a broadcast → FLOOD rule (for ARP, safe because this topology has no loop) and one `eth_dst = host MAC → port` rule per host (priority 10, permanent).
- Both count Packet-In messages. IPv6 is dropped in both so background IPv6 traffic from the hosts does not distort the count.

Experiment and results

Same order for both controllers, starting from an empty network: `dump-flows s1` → `h1 ping -c 5 h5` (first traffic) → `pingall` → `dump-flows s1`, `dump-flows s2`. Main measure: **number of Packet-Ins**; secondary: first-ping RTT.

Measure	Reactive	Proactive
Packet-Ins handled by the controller (ping + pingall)	**52**	**0**
Rules on s1 before any traffic	2	8
Rules on s1 after pingall	16 (14 learned)	8
First ping h1 → h5	46.8 ms	23.7 ms
Later pings	≈ 10.3 ms	≈ 10.3 ms
pingall loss	0 %	0 %

Proactive s1 table before any traffic (from `Part 1/proactive_mininet.txt`). All rules already exist, so no packet needs the controller:

```
priority=100,ipv6 actions=drop
priority=10,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
priority=10,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"     ... one rule per host (h4, h5 -> s1-eth4)
priority=0 actions=CONTROLLER:65535
```

In the reactive run the s1 table starts with only the table-miss and IPv6 rules and grows to 14 learned `in_port/dl_src/dl_dst` rules after pingall. Every one of them was created by a Packet-In, which the Ryu log shows (`PacketIn #1 … #52`).

Explanation and trade-off

In both cases the first ping takes longer than later ones, partly because of the ARP exchange before the first ICMP packet. The proactive first ping (23.7 ms) is about one extra round trip (ARP). The reactive first ping is a further ~23 ms slower because the ARP and ICMP packets of the first exchange go to the controller (6 Packet-Ins) before rules exist. Once rules exist, forwarding is identical (≈ 10.3 ms, which is 2 × 5 ms link delay plus overhead).

- **Reactive:** setup delay on the first packet of every new flow, and controller load that grows with the number of new flows (52 Packet-Ins here). In return it installs only the rules that are actually used, they expire when idle, and it adapts automatically to hosts it has not seen before.
- **Proactive:** no setup delay and almost no controller load (0 Packet-Ins), but the topology and hosts must be known in advance, rules are installed whether used or not (table space), and it does not adapt by itself when the network changes.

Part 2: Link-metric-based path selection (metric: delay)

Topology: `p2_topo.py`, ring s1–s2–s3–s4, host h*i* on port 1 of s*i*.

Link	s1–s2	s2–s3	s3–s4	s4–s1
Bandwidth	2 Mbit/s	2 Mbit/s	1 Mbit/s	1 Mbit/s
Delay (default / changed test)	**20 / 40 ms**	10 ms	20 ms	15 ms
Ports	s1:2 ↔ s2:2	s2:3 ↔ s3:2	s3:3 ↔ s4:2	s4:3 ↔ s1:3

Design (`p2_controller.py`)

- **Switch-level topology:** learned with Ryu's LLDP link discovery (`--observe-links`). Each directed link is stored as `(src_dpid, dst_dpid) → (src_port, dst_port)`. The controller logs *"topology complete: 4 switches, 8 directed links"*; traffic is only sent after that.
- **Host attachment points:** learned from the first ARP/IPv4 Packet-In that arrives on a port that is not an inter-switch port.
- **Link cost:** the link's one-way delay in ms, read from the same JSON file as the topology. Lower delay = lower cost.
- **Path computation:** Dijkstra on the switch graph. For logging, the controller also lists every loop-free candidate path with its cost, so the reason for each decision is visible.
- **Rules:** on the first IPv4 packet of a host pair, `(ipv4_src, ipv4_dst) → out_port` rules (priority 10, idle timeout 30 s) are installed on every switch of the path, downstream switch first, for both directions. The reverse direction uses the reversed path.
- **ARP in a loop:** the ring has a loop, so flooding ARP would circulate forever. The controller answers ARP requests itself when it knows the target (proxy ARP); otherwise it sends the request only out of host-facing ports, never into the ring.
- IPv6 drop rule as in Part 1. `link_costs()` returns the cost map used for every route; Part 3 overrides only this function.

Assumptions: hosts do not move (first learned location is kept); link delays are symmetric; link delays are configuration values (OpenFlow does not report a link's delay), and the controller and the topology read the same file; switch *sN* has dpid N.

Why the path is selected

For h1 → h3 there are two paths. The controller log (`p2_default_ryu.log`, `p2_changed_ryu.log`):

```
default (s1-s2 = 20 ms)                          changed (s1-s2 = 40 ms)
route 10.0.0.1 -> 10.0.0.3 (s1 -> s3)            route 10.0.0.1 -> 10.0.0.3 (s1 -> s3)
  candidate s1-s2-s3 : 20 + 10 = 30 ms <- chosen    candidate s1-s4-s3 : 15 + 20 = 35 ms <- chosen
  candidate s1-s4-s3 : 15 + 20 = 35 ms              candidate s1-s2-s3 : 40 + 10 = 50 ms
```

With default values the path through s2 has the lower total delay (30 ms vs 35 ms) and is chosen. When the s1–s2 delay is raised to 40 ms, the s2 path costs 50 ms, so the controller switches to the path through s4 (35 ms), as expected.

Flow-table evidence (rule for 10.0.0.1 → 10.0.0.3)

Switch	Default	Changed
s1	`nw_src=10.0.0.1,nw_dst=10.0.0.3 actions=output:2` (to s2)	`… actions=output:3` (to s4)
s2	`… actions=output:3` (to s3)	no rule
s4	no rule	`… actions=output:2` (to s3)
s3	`… actions=output:1` (to h3)	`… actions=output:1` (to h3)

Performance measure: ping RTT h1 ↔ h3

	Path	Path delay	Expected RTT	Measured RTT (pings 2–5)	First ping	h3 → h1	pingall
Default	s1-s2-s3	30 ms	60 ms	**60.3 ms**	≈ 105 ms	60.3 ms	0 % loss
Changed	s1-s4-s3	35 ms	70 ms	**70.2 ms**	≈ 108 ms	70.4 ms	0 % loss

The measured RTT matches twice the computed path delay, so traffic really follows the selected path. The first ping is higher because it includes ARP resolution and rule installation by the controller. In the changed test, keeping the old path through s2 would have given about 100 ms RTT; the controller's choice gives 70 ms.

Part 3: Congestion-aware routing

Design (`p3_controller.py`, subclass of the Part 2 controller)

- **Measuring utilization:** every 1 s the controller sends an OpenFlow port-statistics request to every switch. For directed link u→v it takes the `tx_bytes` difference on u's port towards v, divided by the switch-reported time difference, giving a rate in Mbit/s. Utilization = rate / link bandwidth (bandwidth from the JSON file).
- **Congestion rule:** a directed link above **70 %** utilization is congested and its cost becomes *delay + 100 ms*. Otherwise the cost is the Part 2 delay. The controller logs when a link becomes congested and when it clears.
- **New flows only:** costs are evaluated when a new flow arrives. Existing flows keep their path.
- **Per-flow rules:** Part 2 rules matched only on `(ipv4_src, ipv4_dst)`, so a later flow between the same hosts would reuse the old path. Part 3 matches `ip_proto + IPs + TCP/UDP ports`, so every new transport flow gets its own routing decision with the current measurements. Rules have idle timeout 30 s, so they are removed after the flow ends.
- **Static baseline:** the same controller with `CONGESTION_AWARE=0`. Only the penalty is disabled, so it routes exactly like Part 2 (delay only), with the same per-flow rules and monitoring. This makes the comparison fair.

Traffic scenario

- t = 0 s: **flow A**, h1 → h3, UDP 1.5 Mbit/s for 30 s. The network is idle, so it takes the lowest-delay path s1-s2-s3 and loads s1→s2 and s2→s3 to ≈ 81 % (above the threshold).
- t = 6 s: **flow B**, h1 → h3, TCP for 15 s. This is a new flow, routed with the current measurements.
- t = 11 s: TCP/UDP rules on s1–s4 are dumped. Throughput is measured at the receiver (iperf servers on h3).

Controller decision for flow B

```
aware  (p3_aware_ryu.log)                                  static  (p3_static_ryu.log)
util: s1->s2 1.62Mb/s 81%  s2->s3 1.62Mb/s 81%             util: s1->s2 1.62Mb/s 81%  s2->s3 1.62Mb/s 81%
NEW FLOW TCP 10.0.0.1:51426 -> 10.0.0.3:5002               NEW FLOW TCP 10.0.0.1:51340 -> 10.0.0.3:5002
  congested links: s1->s2 82%, s2->s3 81% (+100 ms each)     congested links: ... (ignored: static mode)
  candidate s1-s4-s3 : 15 + 20 = 35 ms   <- chosen           candidate s1-s2-s3 : 20 + 10 = 30 ms <- chosen
  candidate s1-s2-s3 : 120 + 110 = 230 ms                    candidate s1-s4-s3 : 15 + 20 = 35 ms
```

Both controllers measure the same congestion. The static controller ignores it and puts B on the shorter-delay path, which A already fills. The congestion-aware controller adds 100 ms to the two congested links, which makes the s2 path cost 230 ms, so B is placed on the idle s4 path (35 ms). Flow A is not moved.

Flow-table evidence (t = 11 s)

Switch	Static	Congestion-aware
s1	UDP A → output:2, TCP B → output:2 (both to s2)	UDP A → output:2 (s2), **TCP B → output:3 (s4)**
s2	UDP A and TCP B	UDP A only
s4	no TCP/UDP rules	TCP B only
s3	UDP A and TCP B → output:1 (h3)	UDP A and TCP B → output:1 (h3)

Performance comparison (measured at the receiver)

Measure	Static routing	Congestion-aware
Path of new flow B	s1-s2-s3 (shared with A)	s1-s4-s3
Throughput of flow B (TCP)	**0.37 Mbit/s**	**0.87 Mbit/s**
Flow A (UDP) loss	9.8 %	0 %

With static routing, B competes with A for the 2 Mbit/s links, where A already uses 1.5 Mbit/s, so B gets only about 0.37 Mbit/s and A also loses packets in the shared queue. With congestion-aware routing B uses the idle 1 Mbit/s path and reaches about 0.87 Mbit/s (close to that link's capacity), 2.4× more, while A is not disturbed. In repeated runs B always got 0.85–0.88 Mbit/s with the aware controller and 0.17–0.42 Mbit/s with the static one.

Limitations

- **Existing flows are not rerouted.** Flow A stays on its path even when the network changes.
- **No damping.** The rule uses a single 1 s sample and a hard threshold. A flow whose rate moves around 70 % could make the decision for new flows flip back and forth.
- **Rerouted traffic can congest the other path.** Flow B itself fills s1→s4 (≈ 97 %). A third flow would find both paths congested and would get no benefit from the penalty.
- **Measurement environment.** In some runs (both modes) one 5 s interval of flow A shows a short loss burst. The switch counters stayed constant during it, the topology did not change and no ARP happened, so it comes from Mininet host processes being briefly paused inside WSL2, not from routing. We therefore compare whole-run averages.